

Implémentation d'un système de navigation



Année universitaire
2025-2026



- ▶ **Routage**
- ▶ **Configuration des routes**
- ▶ **Routes paramétrées**
- ▶ **Le service Router**
- ▶ **Configuration de child root**

Routage



Routage - Définition



Le routage est le fait de naviguer d'une vue à une autre selon la demande de l'utilisateur.

Un utilisateur peut:

- Taper l'url d'une vue dans le navigateur
- Cliquer sur un lien, bouton, etc..
- Cliquer sur les boutons de retour du navigateur

Routage - @angular/router



Le package @angular/router implémente **Angular Router**, le système de navigation officiel du framework. Il permet :

- La navigation d'une vue (ou composant) à une autre.
- La gestion centralisée des routes via des objets Route.
- L'utilisation de directives intégrées comme routerLink, router-outlet et routerLinkActive.
- Le support natif des **standalone components**, éliminant le besoin de NgModule.

Routage - <base href>



- Les applications routées mentionnent au routeur comment composer l'url de navigation.
- Ceci est fait en utilisant la balise <base href> à placer comme premier nœud de la balise <head> dans le fichier index.html
- Si le répertoire app est la racine de l'application, il faut écrire:

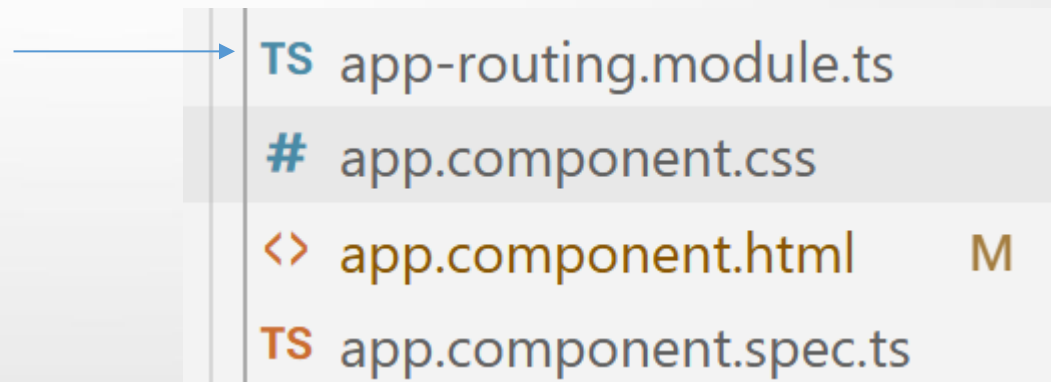
```
<base href="/">
```



Configuration des routes

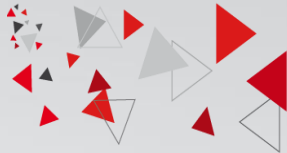
► Configuration des routes – app-routing.module.ts

- Les routes sont définies au niveau d'un module de routage.
- Chaque module possède son propre module de routage
- Le module AppRoutingModuleModule est le module de routage pour le module racine.





Configuration des routes - RouterModule



- Le module RouterModule permet de configurer les routes et propose deux méthodes:

Méthode	Rôle	Lieu de déclaration	Déclaration
forRoot	Créer un module qui contient toutes les directives, les routes et le service router.	Module de routage du module racine «AppRoutingModule»	<pre>@NgModule({ imports: [RouterModule.forRoot(routes, { options }] }) export class AppRoutingModule { }</pre>
forChild	Créer un module qui contient toutes les directives, les routes mais pas le service router.	Module de routage de Features Modules	<pre>@NgModule({ imports: [RouterModule.forChild(routes)]}) export class FeatureRoutingModule { }</pre>

► Configuration des routes



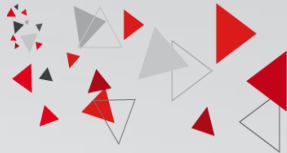
- On importe les éléments nécessaires depuis @angular/router

```
import { RouterModule, Routes } from '@angular/router';
```

- Chaque route associe un chemin (**path**) à un composant via la propriété **component**.
- Le paramètre routes définit un tableau de routes et est défini comme suit avant @NgModule

```
export const routes: Routes = [  
  { path: 'first', component: FirstComponent },  
  { path: 'second', component: SecondComponent },  
];
```

► Configuration des routes



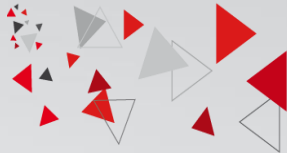
- Le path peut être associé à une url vide pour rediriger l'utilisateur s'il ne tape rien à un composant particulier. Il faut ajouter ainsi la propriété **pathMatch: 'full'** pour informer angular de faire la correspondance sur tout le mot

```
{path: '', redirectTo: 'home', pathMatch: 'full'},  
{ path: 'home', component: HomeComponent },
```

- Pour envoyer des informations static et spécifique à la route appelée (exemple: titre de la page) nous utilisons la propriété "data"

```
{path: 'contact', component: ContactComponent, data: { title: 'Page de contact' }}
```

► Configuration des routes: wildcard



- Le path peut être associé à une url '**' pour désigner n'importe quelle url tapée

```
{path: '**', component: HomeComponent}
```

- La route ayant comme path:** doit être **en dernière position**. Si le router ne trouve aucun path correspondant au path tapé dans l'url jusqu'au qu'il arrive à cette route alors il appliquera cette route.



L'ordre de la déclaration des routes est important



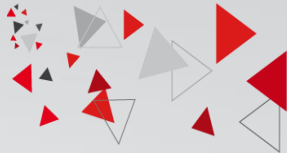
Chargement des routes – RouterOutlet



- `<router-outlet>` est une **directive structurelle** fournie par `@angular/router` qui définit l'emplacement où s'affiche le composant de la route active.
- Définie dans `RouterModule`
- Elle s'utilise dans le composant principal, généralement dans `app.component.ts`.

```
<router-outlet></router-outlet>
```

▶ Création de lien - RouterLink



- les liens HTML natifs:

Un lien classique `<a href="...">` déclenche une requête HTTP vers le serveur, ce qui recharge toute l'application.

- Solution Angular : **routerLink**

La directive `routerLink` permet de naviguer entre les routes Angular sans rechargement de page. Elle intercepte le clic et met à jour l'URL via le routeur Angular.

```
<a routerLink="/home">Accueil</a>
```

▶ Création de lien dynamique



- [routerLink] utilise le 'property binding' pour lier dynamiquement une donnée du composant à l'attribut HTML.
- Dans le fichier TS

```
route = '/search';  
routeName = 'Recherche';
```

- Dans le fichier HTML

```
<a [routerLink]="route">{{ routeName }}</a>
```



Création de lien - RouterLinkActive



- La directive **routerLinkActive** ajoute automatiquement une classe CSS au lien lorsque la route correspondante est active, et la retire sinon.
- Elle Permet de styliser le lien actif (par exemple : soulignement, couleur, etc.)
- Elle Nécessite **d'être importée** comme les autres directives Angular Router

```
<nav>
<a routerLink="/path1" routerLinkActive="active">Accueil</a>
  <a routerLink="/path2" routerLinkActive="active">About</a>
</nav>

<router-outlet></router-outlet>
```




Les routes paramétrées

▶ Les routes paramétrées



Une route paramétrée permet de passer une valeur dynamique via l'URL, comme un identifiant.

- ◆ Déclaration de la route (.ts)

```
{ path: 'produit/:id', component: ProduitComponent }
```

(un paramètre est précédé par « : »)

- ◆ Lien avec paramètre (.html)

```
<a [routerLink]="['/produit', id]">Voir produit</a>
```

id est la valeur de l'id à envoyer dans l'url et récupéré au niveau de l'html



ActivatedRoute



- **ActivatedRoute** est un service qui permet à un composant d'accéder aux informations de la route en cours.
- Il permet de récupérer : Les paramètres de l'URL (paramMap) Les query params (queryParams) Les données statiques (data)...
- Le service **Activated Route** est importé depuis **@angular/router** et injecté au niveau du composant/service dans lequel on veut l'utiliser.

```
import {ActivatedRoute} from '@angular/router';  
...  
constructor( private activatedroute : ActivatedRoute) { }
```

▶ ActivatedRoute - Propriétés



Propriété	Description	Type de retour
snapshot	Retourne une capture sur la route initiale	<u>ActivatedRouteSnapshot</u>
url	Retourne les segments d'URL correspondant à cette route.	Observable<UrlSegment[]>
params http://localhost:4200/profiles/ 5	Retourne un objet contenant les paramètres de cette route indexés par leurs noms.	Observable<Params>
queryParams http://localhost:4200/profiles? id=5	Retourne un objet contenant les query parameters de cette route indexée par leurs noms	Observable<Params>
data	Retourne les données statiques	Observable<Data>
paramMap	Retourne les paramètres obligatoires et facultatifs spécifiques à l'itinéraire. On peut récupérer depuis la valeur de chaque paramètre.	Observable<ParamMap>
queryParamMap	Retourne les paramètres de requête disponibles pour toutes les routes. On peut récupérer depuis la valeur de chaque paramètre.	Observable<ParamMap>

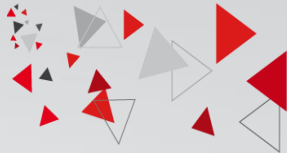
RQ: il y a encore d'autres propriétés

Voir le lien : <https://angular.dev/api/router/ActivatedRoute>



Le service Router

Router - Présentation



- Le Router est un service qui fournit une navigation parmi les vues et des capacités de manipulation d'URL
- Il possède des propriétés qui permettent de suivre l'état de chaque route (routerState, errorHandler, navigated, ...)
- Il possède des méthodes pour manipuler les URLs (createUrlTree(), navigateByUrl(), navigate(),....)
- Il faut l'injecter dans un composant pour l'utiliser
- Provided in dans RouterModule et RouterTestingModule



Router - Utilisation



- Importer le service Router depuis @angular/router

```
import { ActivatedRoute, Router } from '@angular/router';
```

- l'injecter dans le composant/services

```
constructor(private _routes:Router) {}
```

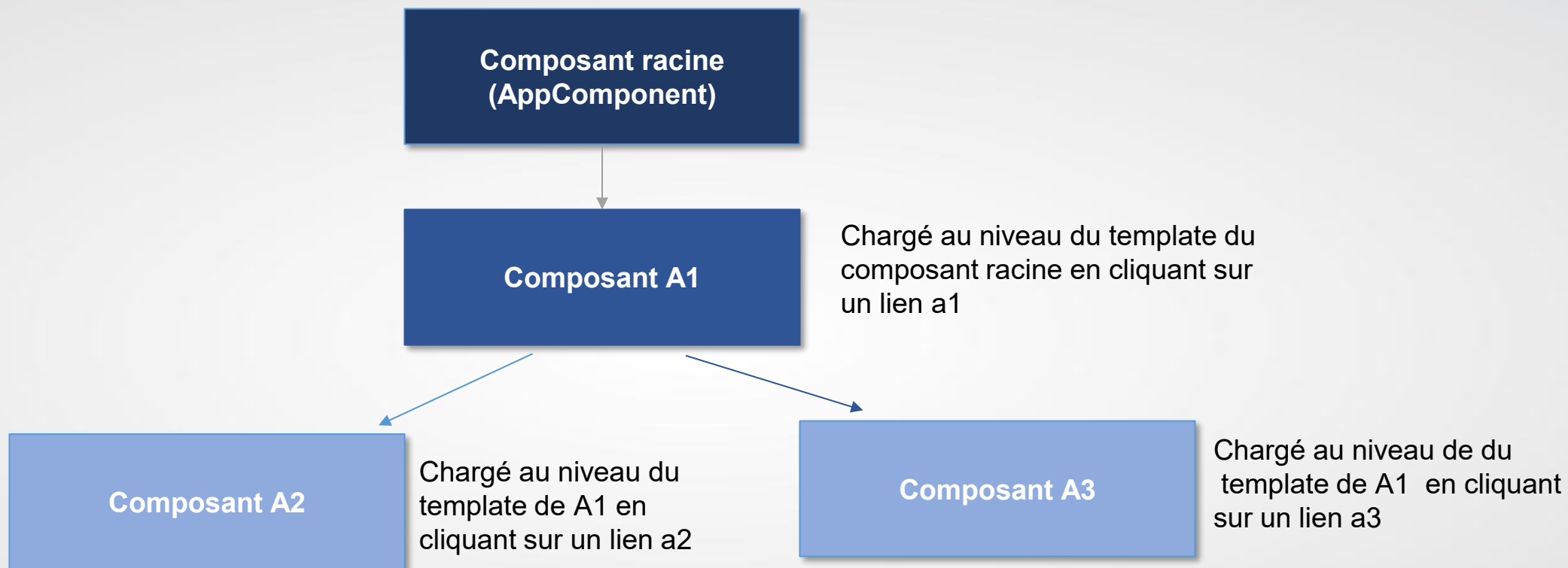
- Exploiter ses propriétés et ses méthodes

```
this._router.navigate(['/profiles', id], {queryParams: {}})
```

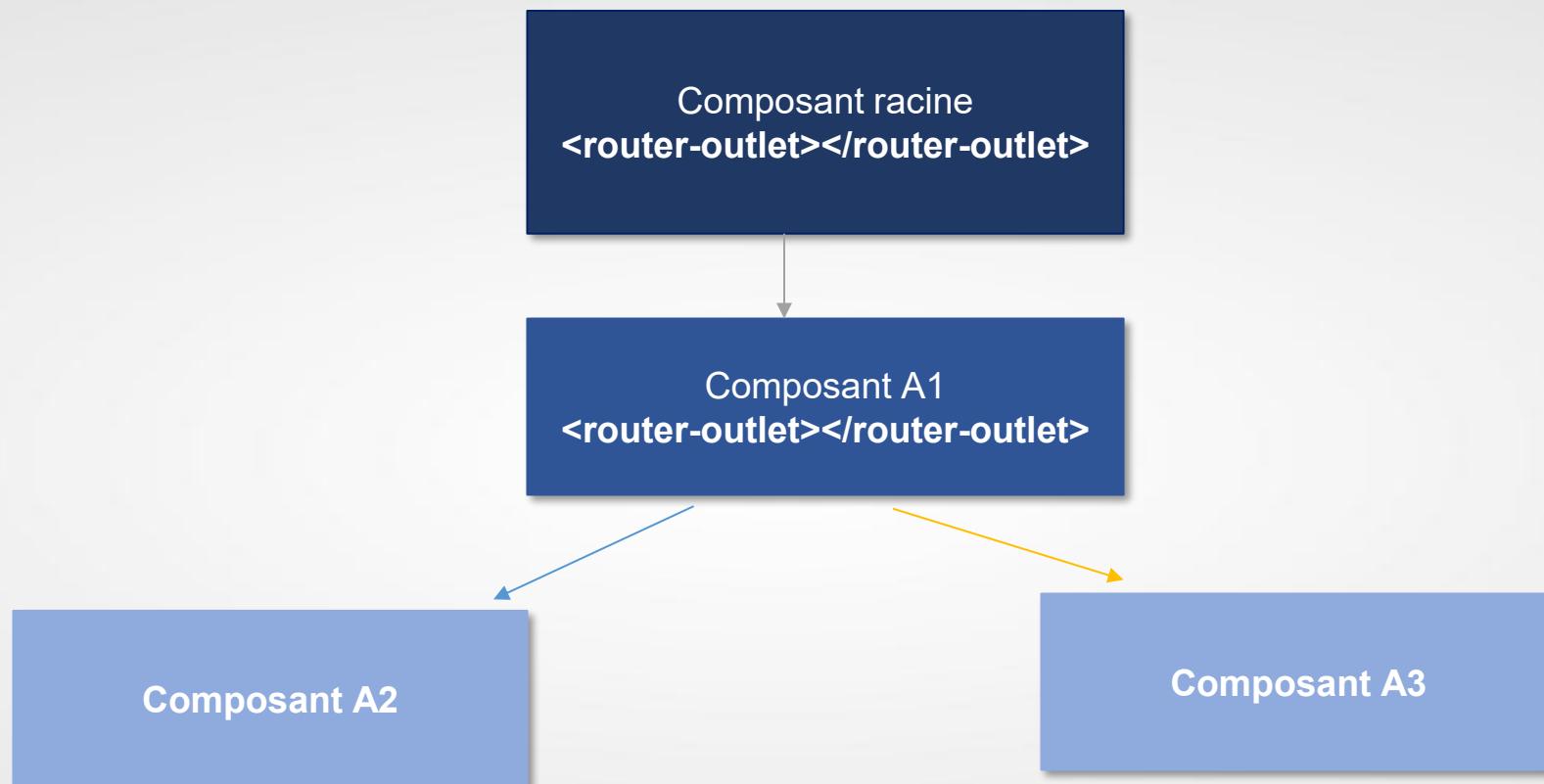


Configuration de child root

► Configuration de « child routes »



► Configuration de « child routes »



► Configuration de « child routes »



```
export const routes: Routes = [  
  
  {path: 'project', component: ProjectComponent, A1  
    children: [  
      A2 {path: 'details', component: DetailsComponent},  
      A3 {path: 'tasks', component: TasksComponent}  ]},  
  
];
```

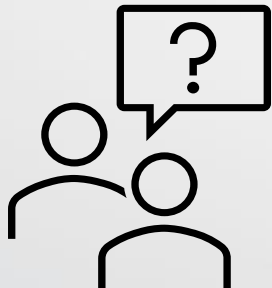
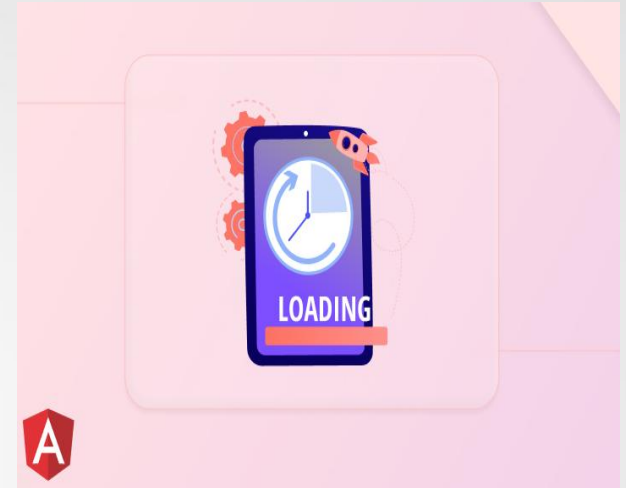


Lazy Loading

► Problème de performance!



- Dans une application Angular, tout le code (composants, pages, images...) est regroupé dans un seul fichier principal : **main.ts**.
- Plus l'application grandit, plus ce fichier devient **lourd** et le **temps de chargement augmente**.
- Les utilisateurs risquent de **quitter le site** si l'affichage est trop lent.



Que faut-il faire pour diminuer le temps de chargement de l'application?

Solution



La solution proposée par Angular est de ne charger, au départ, que les ressources essentielles pour le démarrage de l'application. Le reste des ressources seront chargées à la demande de l'utilisateur.



Il s'agit du Lazy Loading

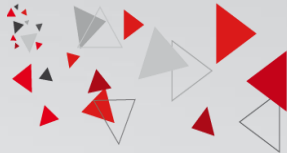
Angular affiche d'abord la partie principale, puis charge le reste **seulement quand on le demande**.

► Principe du Lazy Loading

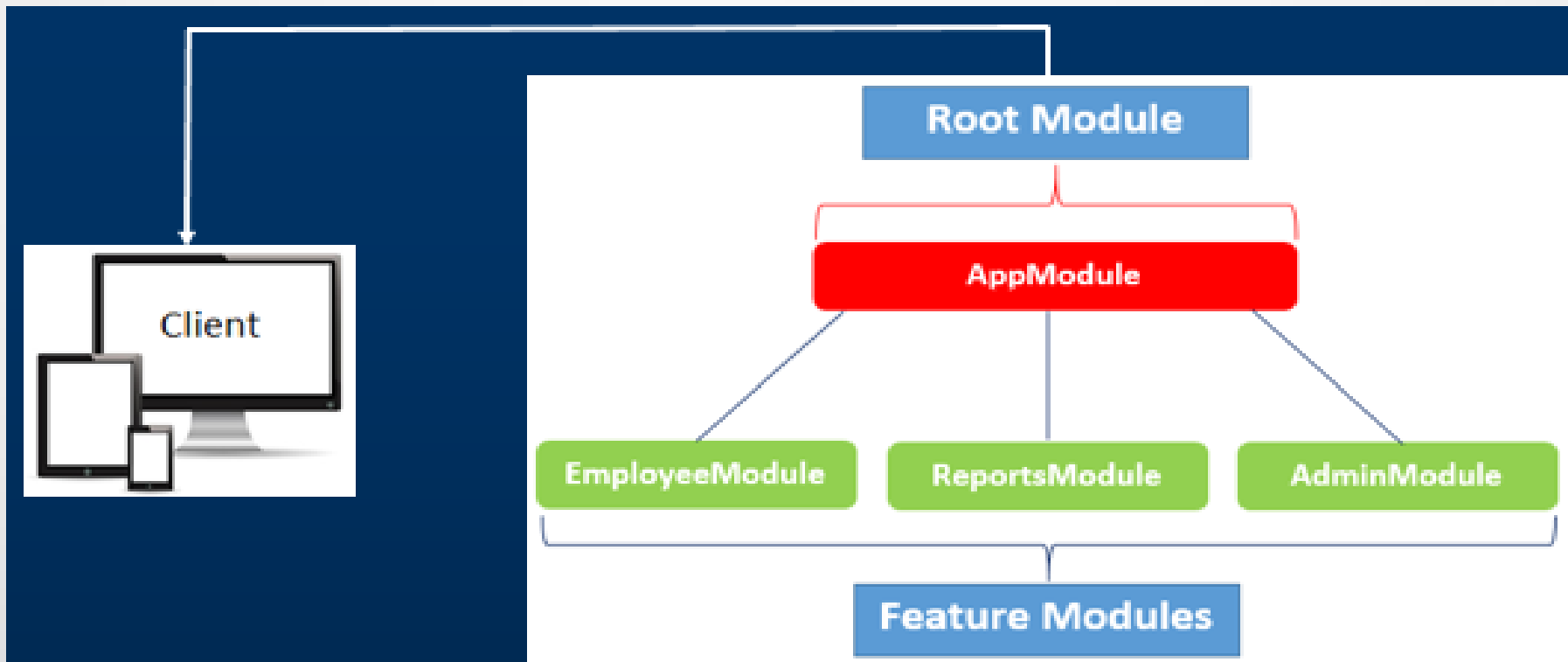


- Le **Lazy Loading** permet de **charger les pages ou parties du site à la demande**.
- Angular affiche d'abord la partie principale, puis charge le reste **seulement quand on le demande**.

► Lazy Loading dans Angular



- En parlant de lazy loading dans Angular, nous parlons de Lazy loaded modules. Ces derniers sont des modules qui sont chargés à la demande lorsque l'utilisateur navigue vers les routes de ces modules.



Avantages



- **Amélioration des performances**

Seules les pages nécessaires sont chargées au démarrage, ce qui **réduit le temps de chargement initial** de l'application.

- **Chargement plus rapide pour l'utilisateur**

L'utilisateur accède rapidement à la partie principale du site, sans attendre le chargement de toutes les fonctionnalités secondaires.

- **Optimisation de la consommation de ressources**

L'application ne charge en mémoire que ce qui est utilisé, ce qui **réduit la consommation RAM** et allège le navigateur.

- **Meilleure organisation du projet**

Chaque fonctionnalité est placée dans un module séparé, ce qui rend le projet **plus structuré, plus maintenable et plus modulaire**.

- **Expérience utilisateur plus fluide**

Le contenu est chargé **à la demande**, ce qui donne une sensation de rapidité et de fluidité.

Etapes



Pour mettre en pratique le lazy loading, il faut:

1. Créer des modules ainsi que leurs modules de routage
2. Associer pour chaque modules les composants correspondants
3. Alimenter les modules de routages des modules créés
4. Mettre à jour le module de routage du module racine en définissant les routes vers ses composants ainsi que les routes vers les modules lazy loaded

► Création de modules



Pour créer un module dans angular vous pouvez lancer la commande:

ng generate module nom_du_module

ou bien son raccourci

ng g m nom_du_module

- Pour créer un module avec son module de routing la commande devient:

ng g m nom_du_module --routing

- Par défaut le module crée n'est pas attaché au module racine. Pour attacher le module créé automatiquement au module racine lors de sa création la commande devient

ng g m nom_du_module --routing --module=app

► Création d'un composant dans un module

Il existe deux méthodes pour créer un composant dans un sous-module

- La première méthode: Se positionner dans le dossier du module concerné puis lancer la commande suivante dans le terminale:

ng g c nom_du_composant

- La deuxième méthode:

ng g c nom_du_module/nom_du_composant

Exemple : ng g c Product/MainProduct

Peu importe la méthode utilisée uniquement le sous-module concerné est mis à jour. Autrement dit le composant créé est ajouté dans la liste des déclarations du module en question, le module racine n'est pas mis à jour.

▶ Exemple

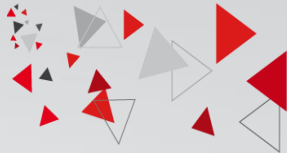


- **Au lancement** → seule la **page d'accueil (App)** est chargée.
- Si l'utilisateur clique sur "**Suggestions**" → Angular charge **SuggestionModule** (chargé uniquement à ce moment).
- Si l'utilisateur clique sur "**User**" → Angular charge **UserModule**.

Résultat :

- L'application démarre plus vite.
- Chaque module est chargé **à la demande**, uniquement quand l'utilisateur visite la page correspondante.

► Mise en place dans le module App



- Dans `app-routing.module.ts`, définir les routes avec Lazy Loading :

```
const routes: Routes = [  
  { path: '', component: HomeComponent },  
  { path: 'suggestions',  
    loadChildren: () => import('./suggestion/suggestion.module')  
      .then(m => m.SuggestionModule) },  
  { path: 'user',  
    loadChildren: () => import('./user/user.module')  
      .then(m => m.UserModule) }];
```

- Le **module App** reste léger car il ne charge que la page courante.
- Les autres pages se chargent **automatiquement quand l'utilisateur les ouvre**.



► **Merci de votre attention**